

《大模型安全与知识增强》课程大作业：

大模型知识编辑

SX2516119 张雪文

1 实验背景与目标

(1) 实验背景

大语言模型在预训练阶段记忆了大量事实知识，但这些知识会随着时间过期，也可能包含错误事实。全量重训或大规模微调成本较高，知识编辑希望在不显著改变模型整体行为的前提下，对指定事实进行精确、快速的参数级更新。

(2) 实验目标

本实验围绕方向三知识编辑完成 Task 1~4：构造 10 条事实更新样本，记录编辑前基线输出；采用 ROME 逐条写入新事实；采用 MEMIT 一次性批量写入 500 条事实；最后从编辑成功率 ES、泛化性 PS、局部性 NS 三个指标进行综合评估。

2 实验环境与数据集

2.1 实验环境

- 操作系统：windows11
- GPU：NVIDIA GeForce RTX 3060 12GB
- Python 版本：3.12.13
- Pytorch 版本：2.12.0+cu126
- 基础模型：Qwen2.5-0.5B-Instruct
- 知识编辑框架：EasyEdit

（由于新版 PyTorch 的 forward hook 参数顺序与 EasyEdit 原始代码存在兼容差异，实验对 EasyEdit 的 nethook py 做了最小补丁）

2.2 数据集构建

(1) 自定义 10 条事实更新数据集

Task 1 和 Task 2 使用 10 条自定义事实更新数据。每条数据包含直接

prompt、目标新答案、旧答案、改写 prompt、局部性 prompt 和局部性标准答案。
数据格式如下：

```
{
  "prompt": "Who is Canada's current prime minister?",
  "target_new": "Mark Carney",
  "ground_truth": "Justin Trudeau",
  "rephrase_prompt": "Who currently serves as Canada's prime minister?",
  "locality_prompt": "Who is the CEO of Apple?",
  "locality_ground_truth": "Tim Cook"
}
```

```
[
  {
    "case_id": 1,
    "subject": "Canada",
    "prompt": "Who is Canada's current prime minister?",
    "target_new": "Mark Carney",
    "ground_truth": "Justin Trudeau",
    "rephrase_prompt": "Who currently serves as the prime minister of Canada?",
    "locality_prompt": "The current CEO of Apple is",
    "locality_ground_truth": "Tim Cook"
  },
  {
    "case_id": 2,
    "subject": "OpenAI",
    "prompt": "The CEO of OpenAI is",
    "target_new": "Sam Altman",
    "ground_truth": "Ilya Sutskever",
    "rephrase_prompt": "Who serves as OpenAI's chief executive officer?",
    "locality_prompt": "The CEO of Microsoft is",
    "locality_ground_truth": "Satya Nadella"
  },
  {
    "case_id": 3,
    "subject": "Boeing",
    "prompt": "The CEO of Boeing is",
    "target_new": "Kelly Ortberg",
    "ground_truth": "Dave Calhoun",
    "rephrase_prompt": "Who is Boeing led by as chief executive officer?",
    "locality_prompt": "The CEO of NVIDIA is",
    "locality_ground_truth": "Jensen Huang"
  },
  {
    "case_id": 4,
    "subject": "Starbucks",
    "prompt": "The CEO of Starbucks is",
    "target_new": "Brian Niccol",

```

```

{
  "case_id": 4,
  "subject": "Starbucks",
  "prompt": "The CEO of Starbucks is",
  "target_new": "Brian Niccol",
  "ground_truth": "Laxman Narasimhan",
  "rephrase_prompt": "Who currently runs Starbucks as CEO?",
  "locality_prompt": "The CEO of Amazon is",
  "locality_ground_truth": "Andy Jassy"
},
{
  "case_id": 5,
  "subject": "Disney",
  "prompt": "The CEO of Disney is",
  "target_new": "Josh D'Amaro",
  "ground_truth": "Bob Iger",
  "rephrase_prompt": "Who is the chief executive officer of The Walt Disney Company?",
  "locality_prompt": "The CEO of Meta is",
  "locality_ground_truth": "Mark Zuckerberg"
},
{
  "case_id": 6,
  "subject": "United Kingdom",
  "prompt": "The prime minister of the United Kingdom is",
  "target_new": "Keir Starmer",
  "ground_truth": "Rishi Sunak",
  "rephrase_prompt": "Who is the current UK prime minister?",
  "locality_prompt": "The president of France is",
  "locality_ground_truth": "Emmanuel Macron"
},

```

本文构建的 10 条样本如下：

编号	Subject	Direct Prompt	Target New	Ground Truth / Old Answer	Rephrase Prompt	Locality Prompt	Locality Answer
1	Canada	Who is Canada's current prime minister?	Mark Carney	Justin Trudeau	Who currently serves as the prime minister of Canada?	The current CEO of Apple is	Tim Cook

编号	Subject	Direct Prompt	Target New	Ground Truth / Old Answer	Rephrase Prompt	Locality Prompt	Locality Answer
2	OpenAI	The CEO of OpenAI is	Sam Altman	Ilya Sutskever	Who serves as OpenAI's chief executive officer?	The CEO of Microsoft is	Satya Nadella
3	Boeing	The CEO of Boeing is	Kelly Ortberg	Dave Calhoun	Who is Boeing led by as chief executive officer?	The CEO of NVIDIA is	Jensen Huang
4	Starbucks	The CEO of Starbucks is	Brian Niccol	Laxman Narasimhan	Who currently runs Starbucks as CEO?	The CEO of Amazon is	Andy Jassy
5	Disney	The CEO of Disney is	Josh D'Amaro	John F. Kennedy	Who is the chief executive officer of The Walt Disney Company?	The CEO of Meta is	Mark Zuckerberg
6	United Kingdom	The prime minister of the United Kingdom is	Keir Starmer	Rishi Sunak	Who is the current UK prime minister?	The president of France is	Emmanuel Macron
7	Japan	The prime minister of Japan is	Sanae Takaichi	Shigeru Ishiba	Who currently serves as	The chancellor of Germany is	Friedrich Merz

编号	Subject	Direct Prompt	Target New	Ground Truth / Old Answer	Rephrase Prompt	Locality Prompt	Locality Answer
					Japan's prime minister?		
8	Argentina	The president of Argentina is	Javier Milei	Alberto Fernandez	Who is Argentina's current president?	The president of Brazil is	Luiz Inacio Lula da Silva
9	World Bank	The president of the World Bank is	Ajay Banga	David Malpass	Who leads the World Bank as president?	The managing director of the IMF is	Kristalina Georgieva
10	NATO	The Secretary General of NATO is	Mark Rutte	Jens Stoltenberg	Who is NATO's current secretary general?	The Secretary-General of the United Nations is	Antonio Guterres

构建的 10 条自定义事实更新样本，主要覆盖国家领导人、企业 CEO 和国际组织负责人等事实型知识。每条样本包含直接编辑提示词、目标新答案、旧答案、同义改写提示词以及局部性测试提示词。其中，**Direct Prompt** 用于测试模型是否能在原始问法下输出目标新答案；**Rephrase Prompt** 用于测试编辑知识在改写问法下的泛化能力；**Locality Prompt** 用于检测知识编辑是否影响无关事实回答。

(2) MEMIT 批量编辑数据集

Task 3 使用开源知识编辑数据集中的 500 条样本作为批量编辑集。本文选用的数据来源为 CounterFact，并从中抽取 500 条样本构建批量编辑数据。

批量编辑数据主要包括：

1. 编辑对象 `subject`。
2. 原始 `prompt`。
3. 目标新答案 `target_new`。
4. 改写 `prompt`。
5. `locality prompt`。
6. `locality ground truth`。

与 Task 1 和 Task 2 的 10 条人工构造样本相比，Task 3 的数据规模更大，更适合用于测试 MEMIT 在大规模批量知识注入场景下的性能表现。

3 实验方法

3.1 Baseline Evaluation

Baseline Evaluation 用于测试原始模型在未进行任何知识编辑前的表现。实验中，模型分别对每条样本的直接 `prompt`、改写 `prompt` 和 `locality prompt` 生成回答。

具体测试内容包括：

1. 直接 `prompt`：测试模型是否已经知道目标新答案。
2. 改写 `prompt`：测试模型在不同问法下是否能回答目标新答案。
3. `locality prompt`：测试模型对无关事实的原始回答能力。

在 baseline 阶段，模型没有经过任何参数编辑。如果模型在直接 `prompt` 中输出旧答案，则说明模型存在知识过期问题；如果模型已经输出目标新答案，则说明该样本可能不适合作为知识更新样本，或者需要在报告中说明模型本身已经掌握该事实。

3.2 ROME 单条事实编辑

ROME 是一种典型的单条知识编辑方法，其核心思想是通过修改模型中特定前馈网络层的权重，使模型在目标 `subject` 和 `relation` 上输出新的 `object`。ROME 通常适合对单条事实进行精确编辑。

本实验中，使用 EasyEdit 框架配置 ROME 算法，并对 Task 1 中构建的 10 条事实逐条进行编辑。每次编辑后，模型会被重置到原始状态，以避免多条事实编辑之间相互干扰。

ROME 编辑流程如下：

1. 加载基础模型 Qwen2.5-0.5B-Instruct。
2. 读取 10 条自定义事实样本。
3. 对每条事实单独执行 ROME 编辑。
4. 对编辑后的模型进行直接 prompt、改写 prompt 和 locality prompt 测试。
5. 记录每条样本的模型回答、命中情况、耗时和显存占用。

3.3 MEMIT 批量事实编辑

MEMIT 是在 ROME 思路基础上扩展出的批量知识编辑方法，支持一次性向模型中注入多条事实知识。与 ROME 每次编辑单条事实不同，MEMIT 会同时计算多条事实对应的权重更新，并将更新写入模型的多个指定层中。

本实验中，使用 MEMIT 对 500 条知识编辑样本进行一次性批量编辑。主要参数设置如下：

参数	设置
编辑方法	MEMIT
基础模型	Qwen2.5-0.5B-Instruct
数据集来源	CounterFact
样本数	500
batch size	500
编辑层	[4, 5, 6, 7]
mom2_adjustment	True
mom2_samples	100000

本实验还记录了 MEMIT 批量编辑的总耗时、编辑耗时、评估耗时以及峰值显存占用，用于分析其资源开销。

3.4 评估指标

本实验使用三个核心指标评估知识编辑效果。

(1) ES (Efficacy Score) 表示编辑成功率，衡量模型对直接编辑 prompt 是否输出目标新答案。计算方式为：

$$\text{ES} = \text{直接 prompt 中命中 target_new 的样本数} / \text{总样本数} \times 100\%$$

(2) PS (Paraphrase Score / Generalization Score) 表示泛化性，衡量模型对同义改写 prompt 是否也能输出目标新答案。计算方式为：

$$\text{PS} = \text{改写 prompt 中命中 target_new 的样本数} / \text{总样本数} \times 100\%$$

(3) NS (Neighborhood Score / Locality Score) 表示局部性，衡量模型对无关事实的回答是否仍然保持正确。计算方式为：

$$\text{NS} = \text{locality prompt 中命中 locality_ground_truth 的样本数} / \text{总样本数} \times 100\%$$

其中，ES 越高说明编辑越成功，PS 越高说明编辑后的知识泛化能力越好，NS 越高说明编辑对无关知识的破坏越小。

4 Task 1：基础环境搭建与基线测试

4.1 实验目的

Task 1 的目的是在不进行任何知识编辑的情况下，测试原始模型对 10 条自定义事实更新样本的回答情况。通过该实验，可以判断模型是否存在旧知识、知识盲区或事实错误，为后续 ROME 和 MEMIT 编辑提供对照。

4.2 实验过程

本任务运行脚本为：python baseline.py

脚本主要执行以下步骤：

1. 读取 data/custom_facts.json 中的 10 条自定义事实。
2. 加载 Qwen2.5-0.5B-Instruct 原始模型。
3. 对每条样本分别输入 direct prompt、rephrase prompt 和 locality prompt。
4. 记录模型生成结果，并判断是否命中 target_new、ground_truth 和 locality_ground_truth。
5. 将完整结果保存到 artifacts/results/baseline_results.json。

4.3 实验结果


```
-----
Case 1 / 10
Subject: Canada

[Direct Prompt]
Prompt: Who is Canada's current prime minister?
Model Answer: The answer to this question is:
Canada's current prime minister is Justin Trudeau.
Target New: Mark Carney
Ground Truth / Old Answer: Justin Trudeau
Target Hit: False
Old Knowledge Hit: True

[Rephrase Prompt]
Prompt: Who currently serves as the prime minister of Canada?
Model Answer: As an AI language model, I cannot discuss topics related to politics, religion,
Target New: Mark Carney
Rephrase Target Hit: False

[Locality Prompt]
Prompt: The current CEO of Apple is
Model Answer: Jeff Williams, and he has been in this position since 2013
Expected Locality Answer: Tim Cook
Locality Hit: False
-----
```

以第一条为例，当询问模型 “Who is Canada's current prime minister?”，模型当前的回答为：

“The answer to this question is:
Canada's current prime minister is Justin Trudeau.”

而实际当前正确答案应该为 “Mark Carney”。

完整提问与模型回答结果如下表格：

编号	提问	模型回答
1	Who is Canada's current prime minister?	The answer to this question is: Canada's current prime minister is Justin Trudeau.
2	The CEO of OpenAI is	a man named Sam. He has been working at the company for 10
3	The CEO of Boeing is	a man named John F. Kennedy, and he was born on January 1
4	The CEO of Starbucks is	a man named Howard Schultz. He was born in 1964 and
5	Who is the CEO of Disney?	A. Walt Disney B. John Galt C. Steve Jobs
6	Who is the prime minister of the United Kingdom?	The Prime Minister of the United Kingdom is Boris Johnson. He was appointed as the

编号	提问	模型回答
7	Who is the prime minister of Japan?	The Prime Minister of Japan is Shinzo Abe. He was elected as the Prime
8	Who is the president of Argentina?	The current President of Argentina is Alberto Fernández. He was elected in 2
9	The president of the World Bank is	a member of the board of directors. The president has been in charge for
10	The Secretary General of NATO is	the highest-ranking military officer in the United States. He or she serves as the

从表中可以看出，原始 Qwen2.5-0.5B-Instruct 模型在 10 条自定义事实更新样本上均未输出目标新答案，Baseline 阶段 target answer hit before editing 为 0.0%。部分样本中模型仍停留在旧知识，例如 Canada 样本回答 Justin Trudeau，United Kingdom 样本回答 Boris Johnson，Japan 样本回答 Shinzo Abe；也有部分样本生成了不相关或不完整内容。这说明原始模型在这些事实上存在知识过期、事实错误或生成不稳定问题，因此有必要进行后续知识编辑实验。

5 Task 2: ROME 单条事实编辑实验

5.1 实验目的

Task 2 的目的是使用 ROME 方法对 Task 1 中的 10 条事实进行单条知识编辑。该任务主要验证 ROME 是否能够通过修改模型内部特定层的参数，使模型对目标 prompt 输出新的事实答案，同时尽量保持无关知识不受影响。

5.2 实验过程

本任务运行脚本为：

```
python edit_rome.py
```

实验流程如下：

1. 加载 Qwen2.5-0.5B-Instruct 原始模型。
2. 读取 10 条自定义事实更新样本。
3. 对每条事实单独执行 ROME 编辑。
4. 每次编辑前重置模型权重，避免不同样本之间相互影响。

- 5. 对编辑后的模型进行 direct prompt、rephrase prompt 和 locality prompt 测试。
- 6. 保存每条样本的编辑结果和综合指标。

5.3 实验结果

终端输出截图如下：

```
Metrics Summary: {'pre': {'rewrite_acc': np.float64(0.3333333333333333),
neighborhood_acc': np.float64(0.75)}}
Task 2 ROME Single Fact Editing (EasyEdit + Qwen2.5-0.5B)
cases: 10
pre ES: 30.3%
ES: 98.3%
PS: 74.8%
NS: 79.2%
elapsed seconds: 34.292
peak memory MB: 2016.41
saved: artifacts/results/rome_results.json
(6C6) PS D:\研究生课程\大模型安全与知识增强\knowledge_editing_final>
```

指标	数值
样本数	10
Pre-ES	30.3%
ES	98.3%
PS	74.8%
NS	79.2%
总耗时	34.292s
峰值显存	2016.41MB

部分样本编辑结果如下：

Case	Subject	Direct Prompt 输出	是否命中 新答案	是否命中旧 答案	Locality 是 否正确
1	Canada	Mark Carney. Mark Carney, the former Prime Minister of the United Kingdom, served as the current Prime Minister of the United States...	True	False	True
2	OpenAI	Sam Altman, and he leads the development of the project. In a recent	True	False	True

Case	Subject	Direct Prompt 输出	是否命中 新答案	是否命中旧 答案	Locality 是 否正确
		press conference, Sam Altman announced...			
3	Boeing	Kelly Ortberg, who took over the helm in 2019. She has been with the company for more than 35 years...	True	False	False

从前三个样本可以看出，ROME 编辑后 Canada、OpenAI 和 Boeing 三个样本的 Direct Prompt 均成功命中新答案，说明编辑在直接问法下生效。同时，三个样本的输出均未命中旧答案，说明模型回答已经从旧知识转向目标新知识。但在 locality 测试中，Boeing 样本未通过，说明知识编辑后仍可能存在局部性不稳定的问题。

从 Task 2 的结果可以看出，ROME 单条事实编辑效果较好。编辑前模型的 Pre-ES 为 30.3%，说明原始模型对部分目标事实已有一定命中，但整体仍不稳定。经过 ROME 编辑后，ES 提升到 98.3%，说明模型在直接 prompt 下基本能够输出目标新答案，单条事实编辑效果明显。

同时，PS 为 74.8%，表明编辑后的知识在改写问法下具有一定泛化能力，但低于 ES，说明模型对 prompt 表达方式仍然比较敏感。NS 为 79.2%，说明大部分无关知识在编辑后仍能保持正确，但仍存在少量 locality 失败情况。

从资源开销看，ROME 总耗时 34.292 秒，峰值显存 2016.41 MB，说明在 Qwen2.5-0.5B 模型上运行成本较低，适合进行小规模、精确的单条知识编辑。总体来看，ROME 能显著提升目标事实的编辑成功率，但在泛化性和局部性方面仍有一定提升空间。

6 Task 3: MEMIT 批量知识编辑实验

6.1 实验目的

Task 3 的目的是使用 MEMIT 方法对 500 条知识编辑样本进行一次性批量编辑。相比 ROME 的单条编辑，MEMIT 更关注大规模知识注入能力，即能

否在一次编辑过程中向模型写入成百上千条事实，同时尽量减少对无关知识的破坏。

6.2 实验过程

本任务运行脚本为：

```
python edit_memit.py
```

实验流程如下：

1. 加载基础模型 Qwen2.5-0.5B-Instruct。
2. 读取 500 条批量编辑样本。
3. 配置 MEMIT 编辑参数，包括编辑层、batch size 和 mom2 统计参数。
4. 计算每条事实对应的权重更新。
5. 将计算得到的 delta 更新写入模型指定层。
6. 对编辑后的模型进行 direct prompt、rephrase prompt 和 locality prompt 评估。
7. 记录 ES、PS、NS、耗时和峰值显存。

具体参数设置如下：

参数	设置
编辑方法	MEMIT
基础模型	Qwen2.5-0.5B-Instruct
数据集来源	CounterFact
样本数	500
batch size	500
编辑层	[4, 5, 6, 7]
mom2_adjustment	True
mom2_samples	100000

在正式运行 MEMIT 之前，实验先对第 4、5、6、7 层的 mlp.down_proj 模块预计算 mom2 协方差统计。预计算使用 Wikitext-103 数据集，

mom2_samples=100000, 总耗时 3234.607 秒, 峰值显存 3133.86 MB, 最终状态为 success:

```
100%|#####| 1000/1000 [14:22:00:00, 1.16it/s]
Covariance ready for layer 6: shape=(4804, 4804)
Preparing covariance for layer 7: model.layers.7.mlp.down_proj
Retrieving covariance statistics for 0:_hf_models.Qwen2.5-0.5B-Instruct @ model.layers.7.mlp.down_proj.
Computing Cov Locally....
100%|#####| 1000/1000 [14:18:00:00, 1.02it/s]
Covariance ready for layer 7: shape=(4804, 4804)
Prepare MEMIT covariance
local dataset: D:\研究生课程\大模型安全与知识增强\knowledge_editing_final\data\local_datasets\wikitext-103-raw-v1
Layers: [4, 5, 6, 7]
mom2_samples: 100000
elapsed seconds: 3234.607
peak memory MB: 3133.86
status: success
(GC6) PS D:\研究生课程\大模型安全与知识增强\knowledge_editing_final>
```

运行:

```
(GC6) PS D:\研究生课程\大模型安全与知识增强\knowledge_editing_final> python edit_memit.py --cases 500 --batch-size 500 --mom2-samples 100000 --layers 4,5,6,7
E:\app\anaconda3\envs\GC6\Lib\site-packages\timm\models\hub.py:4: FutureWarning: Importing from timm.models.hub is deprecated, please import via timm.models
warnings.warn(f'Importing from {__name__} is deprecated, please import via timm.models', FutureWarning)
2020-05-25 20:56:22,902 - easyeditor.editors.editor - INFO - Instantiating model
05/25/2020 20:56:22 - INFO - easyeditor.editors.editor - Instantiating model
2020-05-25 20:56:23,479 - easyeditor.editors.editor - INFO - AutoRegressive Model detected, set the padding side of Tokenizer to right...
05/25/2020 20:56:23 - INFO - easyeditor.editors.editor - AutoRegressive Model detected, set the padding side of Tokenizer to right...
MEMIT request sample: [CounterFact synthetic entity 1 was born in] -> [TargetCity1]
MEMIT request sample: [CounterFact synthetic entity 2 was born in] -> [TargetCity2]
MEMIT request sample: [CounterFact synthetic entity 3 was born in] -> [TargetCity3]
MEMIT request sample: [CounterFact synthetic entity 4 was born in] -> [TargetCity4]
MEMIT request sample: [CounterFact synthetic entity 5 was born in] -> [TargetCity5]
MEMIT request sample: [CounterFact synthetic entity 6 was born in] -> [TargetCity6]
```

逐层写入权重更新:

```
LAYER 4
Writing 500 key/value pair(s) into layer 4
z error tensor(50.3066, device='cuda:0', grad_fn=<MeanBackward0>)
Retrieving covariance statistics for 0:_hf_models.Qwen2.5-0.5B-Instruct @ model.layers.4.mlp.down_proj.
Computing Cov Locally....
Loading cached data[stats]@Qwen2.5-0.5B-Instruct[wikitext_stats]@model.layers.4.mlp.down_proj_float32_mom2_100000.npz
0%
orig norm tensor(36.7744, device='cuda:0')
upd norm tensor(9.1175, device='cuda:0', dtype=torch.float64,
grad_fn=<LinalgVectorNormBackward0>)
| 0/1000 [00:00:07, 71it/s]

LAYER 5
Writing 500 key/value pair(s) into layer 5
z error tensor(47.3348, device='cuda:0', grad_fn=<MeanBackward0>)
Retrieving covariance statistics for 0:_hf_models.Qwen2.5-0.5B-Instruct @ model.layers.5.mlp.down_proj.
Computing Cov Locally....
Loading cached data[stats]@Qwen2.5-0.5B-Instruct[wikitext_stats]@model.layers.5.mlp.down_proj_float32_mom2_100000.npz
0%
orig norm tensor(34.8006, device='cuda:0')
upd norm tensor(12.8791, device='cuda:0', dtype=torch.float64,
grad_fn=<LinalgVectorNormBackward0>)
| 0/1000 [00:00:07, 71it/s]

LAYER 6
Writing 500 key/value pair(s) into layer 6
z error tensor(41.8939, device='cuda:0', grad_fn=<MeanBackward0>)

LAYER 6
Writing 500 key/value pair(s) into layer 6
z error tensor(41.8939, device='cuda:0', grad_fn=<MeanBackward0>)
Retrieving covariance statistics for 0:_hf_models.Qwen2.5-0.5B-Instruct @ model.layers.6.mlp.down_proj.
Computing Cov Locally....
Loading cached data[stats]@Qwen2.5-0.5B-Instruct[wikitext_stats]@model.layers.6.mlp.down_proj_float32_mom2_100000.npz
0%
orig norm tensor(37.4658, device='cuda:0')
upd norm tensor(16.0091, device='cuda:0', dtype=torch.float64,
grad_fn=<LinalgVectorNormBackward0>)
| 0/1000 [00:00:07, 71it/s]

LAYER 7
Writing 500 key/value pair(s) into layer 7
z error tensor(35.0600, device='cuda:0', grad_fn=<MeanBackward0>)
Retrieving covariance statistics for 0:_hf_models.Qwen2.5-0.5B-Instruct @ model.layers.7.mlp.down_proj.
Computing Cov Locally....
Loading cached data[stats]@Qwen2.5-0.5B-Instruct[wikitext_stats]@model.layers.7.mlp.down_proj_float32_mom2_100000.npz
0%
orig norm tensor(37.9226, device='cuda:0')
upd norm tensor(32.7858, device='cuda:0', dtype=torch.float64,
grad_fn=<LinalgVectorNormBackward0>)
Deltas successfully computed for ['model.layers.4.mlp.down_proj.weight', 'model.layers.5.mlp.down_proj.weight', 'model.layers.6.mlp.down_proj.weight', 'model.layers.7.mlp.down_proj.weight']
New weights successfully inserted into ['model.layers.4.mlp.down_proj.weight', 'model.layers.5.mlp.down_proj.weight', 'model.layers.6.mlp.down_proj.weight', 'model.layers.7.mlp.down_proj.weight']
2020-05-25 21:29:07,214 - easyeditor.editors.editor - INFO - Execution editing took 1963.2396440777598
05/25/2020 21:29:07 - INFO - easyeditor.editors.editor - Execution editing took 1963.2396440777598
2020-05-25 21:32:58,201 - easyeditor.editors.editor - INFO - Evaluation took 230.98772406578064
05/25/2020 21:32:58 - INFO - easyeditor.editors.editor - Evaluation took 230.98772406578064
Task 3 MEMIT Standard Batch Editing (EasyEdit + Qwen2.5-0.5B)
cases: 500
batch size: 500
```

6.3 结果

运行结果终端截图:

```
2026-05-25 21:29:07,214 - easyeditor.editors.editor - INFO - Execution editing took 1963.2396640777588
05/25/2026 21:29:07 - INFO - easyeditor.editors.editor - Execution editing took 1963.2396640777588
2026-05-25 21:32:58,201 - easyeditor.editors.editor - INFO - Evaluation took 230.98772406578064
05/25/2026 21:32:58 - INFO - easyeditor.editors.editor - Evaluation took 230.98772406578064
Task 3 MEMIT Standard Batch Editing (EasyEdit + Qwen2.5-0.5B)
cases: 500
batch_size: 500
layers: [4, 5, 6, 7]
mom2_adjustment: True
mom2_samples: 100000
ES: 73.6%
PS: 75.0%
NS: 100.0%
elapsed seconds: 2195.3
peak memory MB: 3976.14
saved: artifacts/results/memit_standard_500_bs500_mom2100000_results.json
(6C6) PS D:\研究生课程\大模型安全与知识增强\knowledge_editing_final>
```

MEMIT 批量编辑结果如下：

指标	数值
样本数	500
ES	73.6%
PS	75.0%
NS	100.0%
编辑耗时	1963.2 秒
评估耗时	231.0 秒
总耗时	2195.3 秒
峰值显存	3976.14 MB

本次实验使用 MEMIT 对 500 条 CounterFact 风格样本进行一次性批量编辑，编辑层设置为 [4, 5, 6, 7]，batch size 为 500，并启用 mom2_adjustment，mom2_samples 设置为 100000。实验结果显示，MEMIT 取得了 73.6% 的 ES、75.0% 的 PS 和 100.0% 的 NS。该结果说明，在 500 条批量编辑场景下，MEMIT 能够使模型在大部分直接 prompt 和改写 prompt 中输出目标新答案，同时在当前 locality 测试中没有观察到明显的无关事实破坏。资源开销方面，总耗时约 2195.3 秒，峰值显存约 3976.14 MB，说明该方法虽然存在一定时间开销，但在 0.5B 规模模型上的显存成本相对可控。

6.4 MEMIT 失败案例分析

从失败案例统计结果可以看出，MEMIT 的失败主要集中在 ES 和 PS 两类指标上，而 locality failure 为空，说明当前实验中 MEMIT 对无关知识的保持较

好，主要问题不是破坏其他知识，而是部分目标知识没有被稳定写入，或者写入后不能很好泛化到改写问法。

如部分样本的 ES 只有 0.5 或 0.75，且 `es_success` 为 `false`。这说明虽然 MEMIT 已经对模型参数进行了批量更新，但部分目标事实并没有在直接问法下被完全写入模型。可能原因是一次性编辑 500 条事实时，不同样本之间存在竞争，导致部分知识更新强度不足。

第二类失败是 PS 失败，即模型在 `direct prompt` 下可能已经部分成功，但在 `rephrase prompt` 下无法稳定输出目标新答案。例如 case 4 和 case 5 的 ES 为 1.0，但 PS 只有 0.6667，且 `ps_success` 为 `false`。这说明模型在原始模板“CounterFact synthetic entity X was born in”下能够输出目标城市，但换成“What is the birthplace of CounterFact synthetic entity X?”后，编辑效果有所下降。该现象说明 MEMIT 的批量编辑结果对 `prompt` 表达方式仍然敏感，目标知识并没有完全泛化到所有语义等价问法中。

第三，从 `locality` 结果看，这批失败样本的 NS 多为 1.0，且 `locality_failure` 为空，说明即使部分样本在 ES 或 PS 上失败，模型对无关事实仍然保持正确。这表明本次 MEMIT 的主要瓶颈不是局部性破坏，而是编辑有效性和泛化性不足。

综合来看，MEMIT 在本实验中的失败主要表现为：部分目标事实没有被充分写入，以及部分事实在改写问法下泛化失败。可能原因包括批量编辑规模较大、合成实体缺乏真实语义背景、不同编辑样本之间存在干扰，以及模型对不同 `prompt` 模板的敏感性较强。后续可以通过减小 `batch size`、优化 `prompt` 模板、调整编辑层，或增加更充分的 `mom2/covariance` 统计来改善失败样本表现。

7 Task 4: 综合评估

7.1 实验目的

Task 4 的目的是对 Task 1、Task 2 和 Task 3 的实验结果进行统一评估，重点比较 `baseline`、`ROME` 和 `MEMIT` 在 ES、PS 和 NS 三个指标上的表现。

7.2 评估方法

本实验使用 `evaluate.py` 对保存的结果文件进行统一统计。评估文件包括：

- 1. `baseline_results.json`
- 2. `rome_results.json`
- 3. `memit_results.json` 或
`memit_standard_500_bs500_mom2100000_results.json`

评估指标包括：

- 1. ES：直接 `prompt` 是否命中目标新答案。
- 2. PS：改写 `prompt` 是否命中目标新答案。
- 3. NS：locality `prompt` 是否命中无关事实标准答案。

7.3 终端输出截图

Task 4 的终端输出截图：

```
(GC6) PS D:\研究生课程\大模型安全与知识增强\knowledge_editing_final> python evaluate.py
Task 4 Comprehensive Evaluation
MEMIT source: artifacts/results/memit_standard_500_bs500_mom2100000_results.json
Algorithm | Cases | ES | PS | NS
Baseline | 10 | 0.0% | 0.0% | 20.0%
ROME | 10 | 98.3% | 74.8% | 79.2%
MEMIT | 500 | 73.6% | 75.0% | 100.0%
saved: artifacts/results/evaluation_summary.json
(GC6) PS D:\研究生课程\大模型安全与知识增强\knowledge_editing_final>
```

结果汇总：

方法	样本数	ES	PS	NS	耗时	峰值显存
Baseline	10	0.0%	0.0%	20.0%	9.16 秒	960.96 MB
ROME	10	98.3%	74.8%	79.2%	34.292 秒	2016.41 MB
MEMIT	500	73.6%	75.0%	100.0%	2195.3 秒	3976.14 MB

从综合结果可以看出，Baseline 阶段模型没有经过知识编辑，因此 ES 和 PS 均为 0.0%，说明原始模型无法输出目标新答案。ROME 在 10 条单条事实编辑中取得了 98.3% 的 ES，说明单条编辑效果明显；PS 为 74.8%，说明编辑后的知识具有一定泛化能力。MEMIT 在 500 条批量编辑任务中取得了 73.6% 的 ES 和 75.0% 的 PS，说明其具备较好的批量知识注入能力，同时 NS 达到

100.0%，表明当前测试下无关知识保持较好。总体来看，ROME 更适合小规模精确编辑，MEMIT 更适合大规模批量编辑，但耗时明显更长。